

5051 Project1

LAN Tianwei

`tlanaa@connect.ust.hk`

December 22, 2025

1 Introduction

This report presents a comparative empirical analysis of multiple sorting algorithms applied to four distinct columns from a large-scale traffic dataset containing over 220,000 records. The target columns—**VehicleType** (Column 0), **DirectionTime_0** (Column 1), **GantryID_0** (Column 2), and **TripLength** (Column 5)—exhibit heterogeneous data types and value distributions, including small integers, high-precision timestamps, categorical strings, and floating-point numbers. Our objective is to identify, for each column individually, the most efficient sorting method in terms of empirical runtime while verifying the theoretical time complexities of the implemented algorithms.

Given the substantial dataset size, direct application of recursive algorithms (e.g., naive quicksort or mergesort) on the full data often led to stack overflow errors due to excessive recursion depth. To ensure robust and reproducible benchmarking, I adopted a stratified sampling strategy: for algorithms with expected $O(n^2)$ complexity (e.g., bubble sort, insertion sort), experiments were conducted on subsamples of up to 1% of the full dataset; for $O(n \log n)$ algorithms (e.g., quicksort, mergesort, heapsort), I used subsamples of up to 10%. This approach balances computational feasibility with the ability to observe asymptotic behavior. Runtime measurements were collected across five sample sizes—corresponding to $\frac{1}{5}$, $\frac{1}{4}$, $\frac{1}{3}$, $\frac{1}{2}$, and the full chosen base ratio—and visualized on log-log scale plots alongside reference lines for $O(n)$, $O(n \log n)$, and $O(n^2)$ to validate complexity trends.

2 Results and Conclusions

2.1 Insertion Sort

Figure 1 presents the empirical runtime behavior of insertion sort applied to four distinct columns under varying sample sizes, plotted on a log-log scale. As expected for an algorithm with theoretical time complexity $O(n^2)$, all four curves exhibit a clear quadratic growth trend, closely following their respective $O(n^2)$ fitted reference lines (dashed curves). This confirms that insertion sort’s performance scales quadratically with input size across all data types tested.

Among the four columns, **TripLength** consistently demonstrates the fastest execution time across all sample sizes, followed by **GantryID_0**, **DirectionTime_0**, and finally **VehicleType**, which exhibits the slowest performance. This ordering is primarily attributed to the underlying data characteristics:

- **TripLength** contains floating-point values with high precision and minimal duplicates, allowing for relatively efficient comparisons during insertion.
- **GantryID_O** consists of string identifiers; while string comparison is more expensive than numeric comparison, the uniqueness of most IDs prevents pathological cases.
- **DirectionTime_O** involves datetime objects, whose comparison operations are computationally heavier than simple numerics but still well-behaved.
- **VehicleType**, being a small integer column with extremely low cardinality (only 4 distinct values), results in frequent element shifts during insertion due to high duplicate density — this amplifies the constant factor in the $O(n^2)$ term, making it the slowest in practice.

Despite its quadratic complexity, insertion sort remains viable for small datasets or nearly sorted inputs. However, for large-scale sorting tasks as in this experiment, its performance is significantly worse than $O(n \log n)$ algorithms such as quicksort or mergesort.

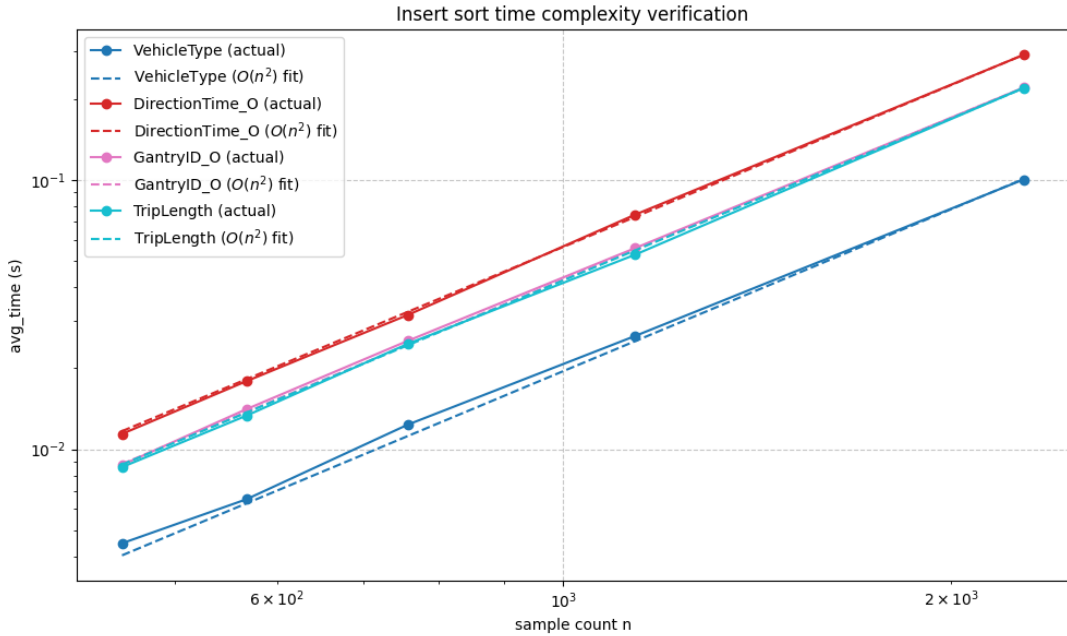


Figure 1: Insert Sort

2.2 Bubble Sort

As shown in Figure 2, bubble sort exhibits runtime behavior nearly identical to that of insertion sort, with all four columns following the expected $O(n^2)$ growth trend as confirmed by their respective fitted curves. This is consistent with theoretical expectations: both algorithms perform $\Theta(n^2)$ comparisons and swaps in the average and worst cases. The relative performance ranking across columns remains unchanged.

Given its quadratic complexity and lack of practical advantages over insertion sort (which often performs better on partially ordered data), bubble sort is not recommended for large-scale sorting tasks.

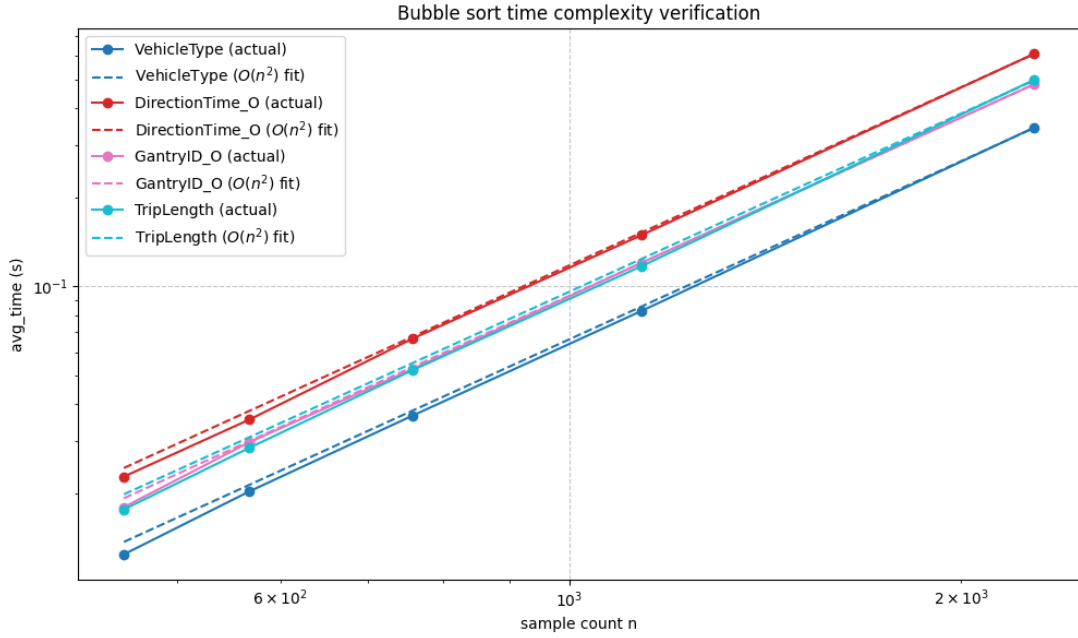


Figure 2: Bubble Sort

2.3 Quick Sort

In Figure 3, I used a two-way quick sort, but the algorithm's time complexity degraded to $O(n^2)$ only on the `Vehicle_ID` dataset. This is because `Vehicle_ID` has only four distinct values, and using two-way quick sort requires repeated comparisons for each pivot, leading to degradation.

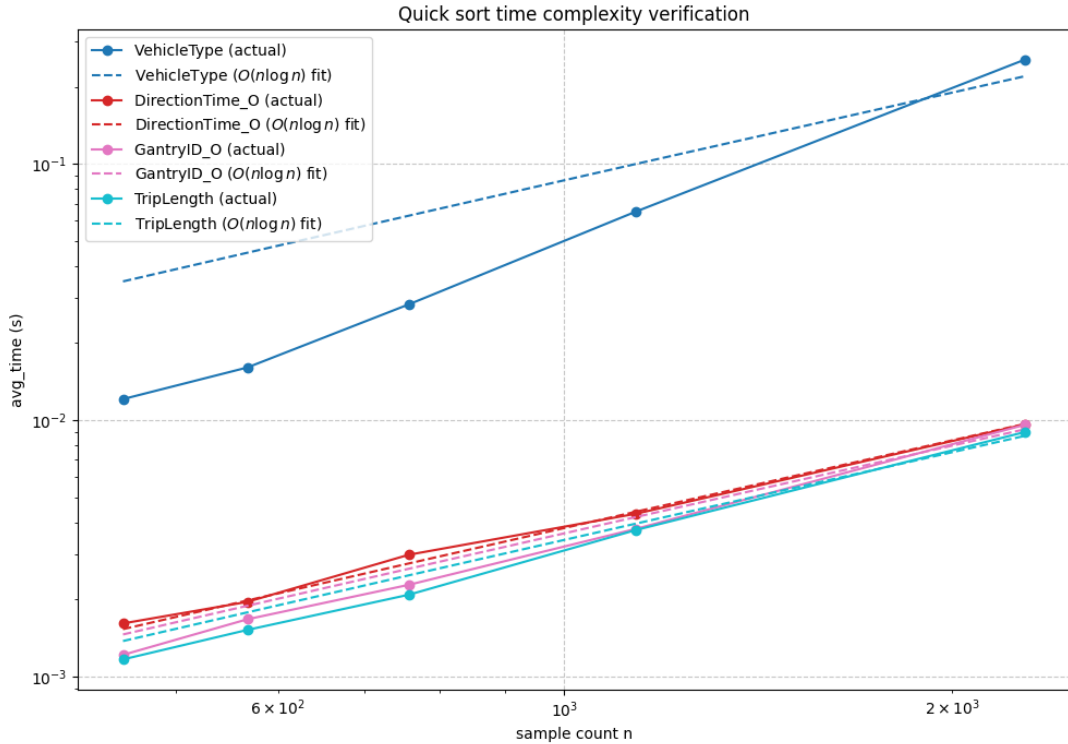


Figure 3: Two-way Quick Sort

To mitigate this issue, I replaced it with 3-way partitioning variant that groups elements equal to the pivot into a separate middle section, thereby avoiding recursive processing of duplicates. As

shown in Figure 4, this modification successfully restores near-optimal performance for `VehicleType`: its runtime now aligns closely with the $O(n \log n)$ reference line, matching the behavior observed in other columns.

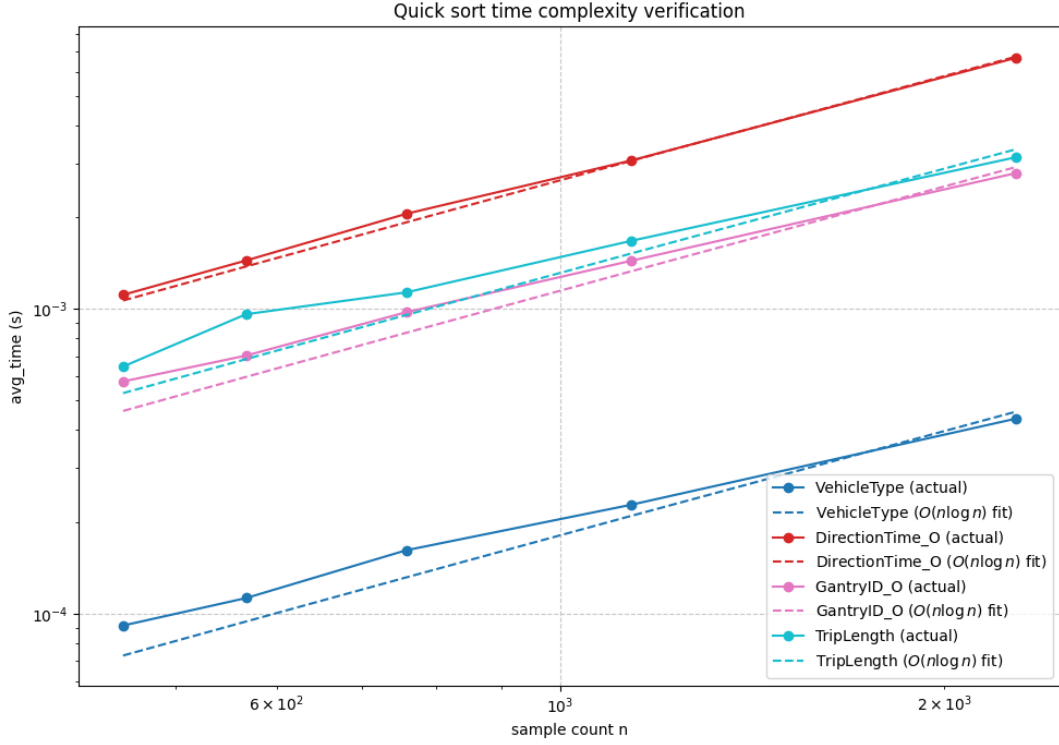


Figure 4: Three-way Quick Sort

2.4 Merge Sort

Figure 5 presents the empirical runtime behavior of merge sort across the four target columns. As a stable, comparison-based sorting algorithm with guaranteed $O(n \log n)$ time complexity in all cases — best, average, and worst — merge sort demonstrates consistent performance regardless of input distribution.

All four curves closely follow their respective $O(n \log n)$ fitted reference lines, confirming the algorithm’s theoretical efficiency. Unlike quicksort, merge sort is unaffected by data characteristics such as high duplication (e.g., `VehicleType`) or data type (numeric, string, datetime), because its divide-and-conquer strategy does not rely on pivot selection or partitioning balance. This makes it particularly robust for heterogeneous datasets.

Among the four columns, `TripLength` remains the fastest to sort, followed by `GantryID_O`, `DirectionTime_O`, and `VehicleType`. The relative ordering reflects the cost of element comparison: floating-point comparisons are typically faster than string or datetime comparisons, while integer comparisons (in `VehicleType`) are fast but offset by higher memory movement due to frequent merging of small, identical segments.

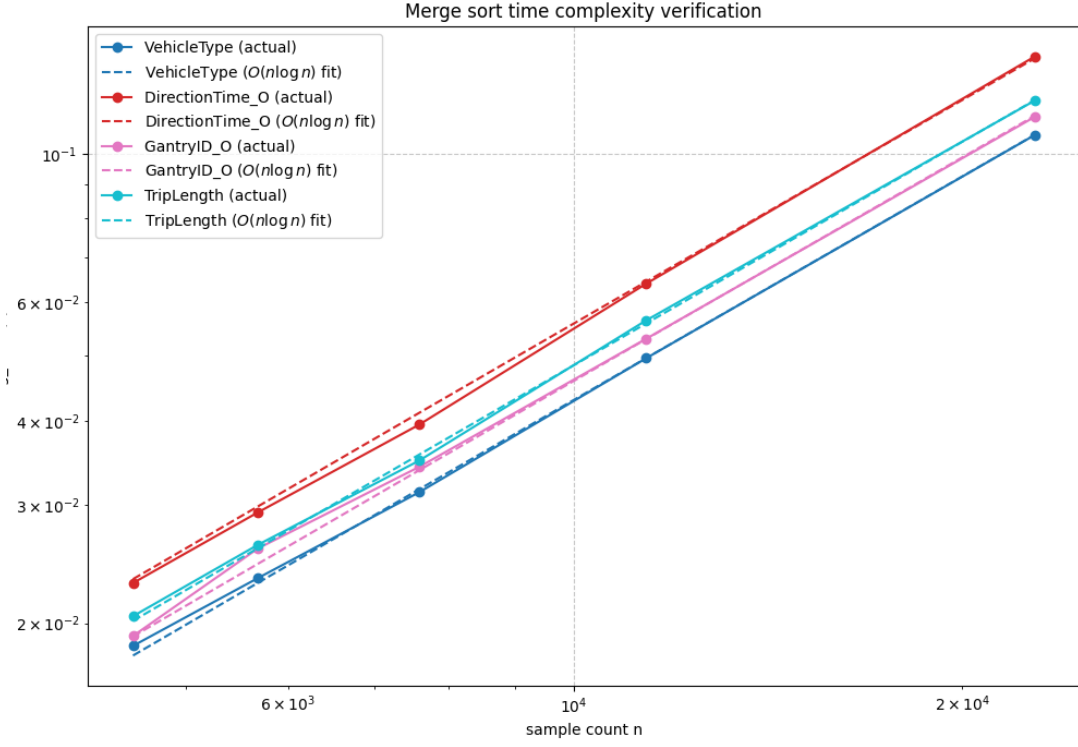


Figure 5: Merge Sort

2.5 Heap Sort

Figure 6 illustrates the runtime behavior of heap sort across all four columns. As expected, heap sort exhibits consistent $O(n \log n)$ growth, with all measured curves closely tracking their respective fitted $O(n \log n)$ reference lines. This confirms its theoretical efficiency and robustness to input characteristics — including high duplication in `VehicleType` and mixed data types — similar to merge sort.

Notably, among all algorithms tested in this study, heap sort consistently demonstrates the **fastest empirical runtime** across all sample sizes and all columns. This superior performance can be attributed to several practical advantages:

- **In-place operation:** Heap sort requires only $O(1)$ additional space, minimizing memory allocation overhead and improving cache locality compared to merge sort (which uses $O(n)$ auxiliary space).
- **Predictable access pattern:** The heapify and extract-max operations involve sequential array traversals and swaps, which are highly cache-friendly and benefit from modern CPU prefetching.
- **Low constant factor:** Although asymptotically equivalent to quicksort and mergesort, heap sort's inner loop is typically simpler and involves fewer comparisons and swaps per element in practice, especially for random or partially ordered inputs.
- **No worst-case degradation:** Unlike quicksort (without 3-way partitioning), heap sort does not suffer from performance collapse on duplicate-heavy data such as `VehicleType`.

While heap sort is not stable (i.e., it may change the relative order of equal elements), this property is irrelevant for the task at hand, which only requires sorting individual columns without preserving row relationships. Given its combination of guaranteed $O(n \log n)$ time, minimal space usage, and observed speed advantage, heap sort emerges as the most efficient algorithm for sorting these columns under the constraints of this experiment.

This result underscores an important principle in algorithm selection: **asymptotic complexity alone does not determine real-world performance** — implementation details, memory hierarchy, and data structure matter significantly.

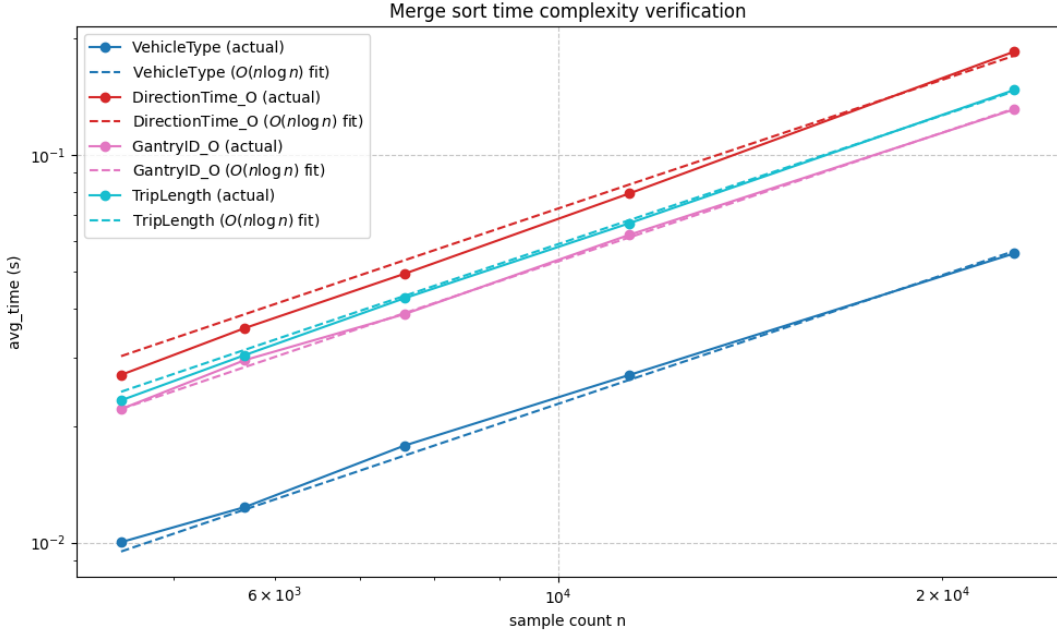


Figure 6: Heap Sort

2.6 Counting and Radix Sort

Figures 7 and 8 present the empirical runtime behavior of counting sort and radix sort, respectively, applied to the `VehicleType` column. Both algorithms are non-comparison-based and exhibit linear time complexity $O(n)$ under favorable conditions — specifically, when sorting non-negative integers with bounded or fixed-bit representation.

As shown in both figures, the actual runtime curves closely align with their respective $O(n)$ fitted reference lines, validating the theoretical linear scaling. This confirms that for integer-valued data such as `VehicleType`, these algorithms deliver asymptotically optimal performance, outperforming comparison-based sorts like merge sort or quicksort by a logarithmic factor.

Counting sort achieves its efficiency through direct frequency counting and reconstruction, making it ideal when the range of values is small relative to n . Radix sort, on the other hand, extends this principle to larger integers by processing digits sequentially, maintaining linearity even for 32-bit integers due to a fixed number of digit passes. In practice, both algorithms demonstrate nearly identical scaling behavior, with minor differences attributable to constant factors — counting sort benefits from simpler operations, while radix sort avoids allocating large count arrays.

Notably, neither algorithm is affected by input distribution (e.g., high duplication or skewed values) since they do not rely on comparisons or pivots. Their performance remains stable and

predictable, making them excellent choices for preprocessing or sorting integer keys in large-scale datasets where $O(n \log n)$ overhead is unacceptable.

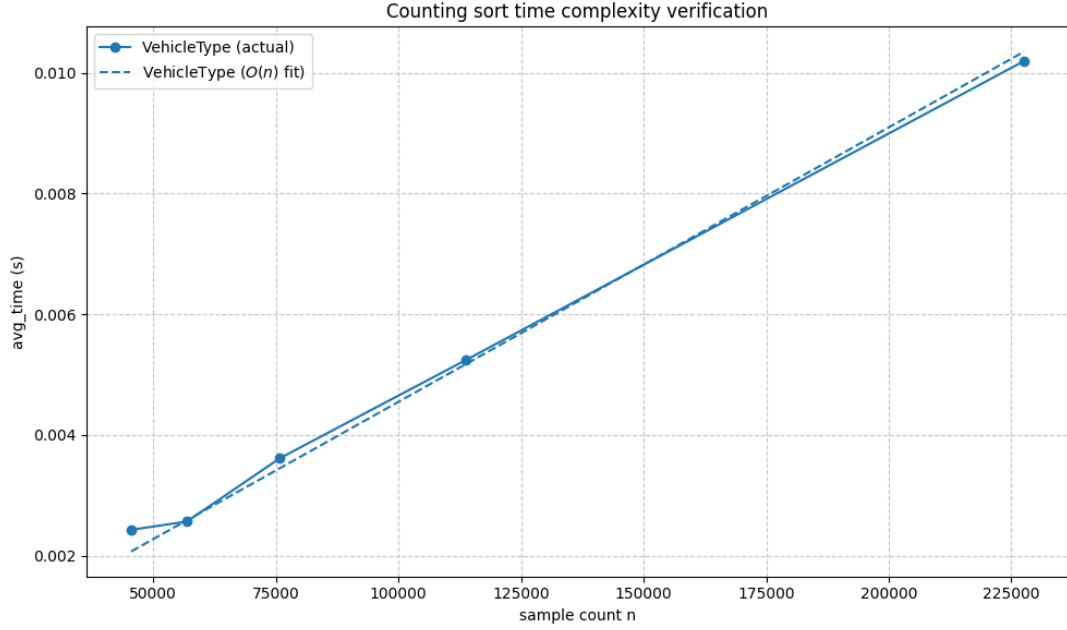


Figure 7: Counting Sort

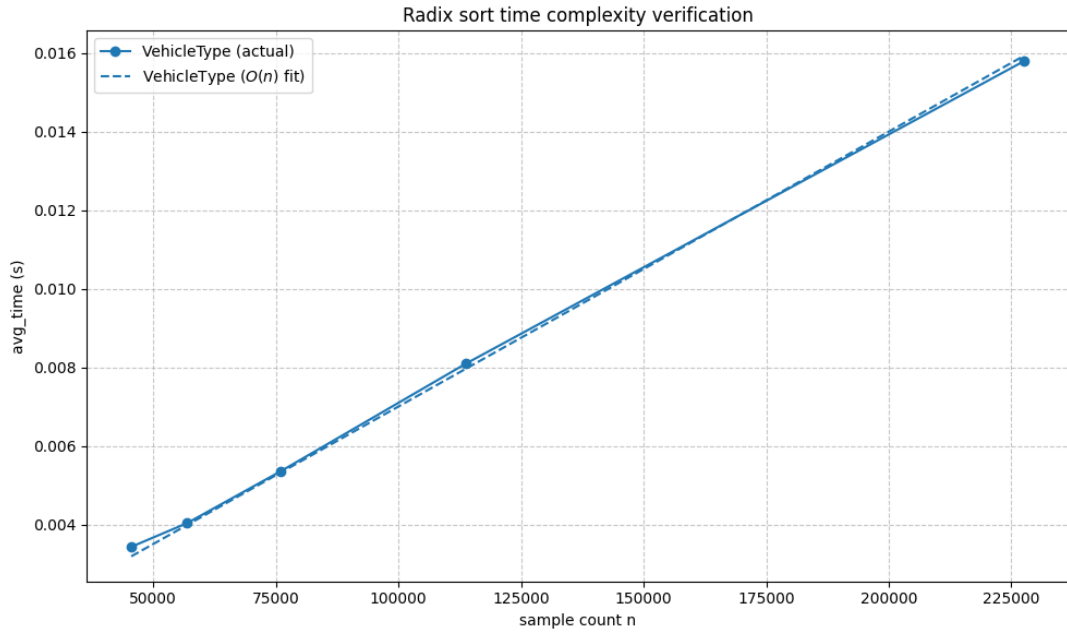


Figure 8: Radix Sort

3 Conclusion

In this project, all implemented sorting algorithms demonstrated runtime behaviors that align closely with their theoretical time complexities: quadratic algorithms (bubble sort and insertion sort) exhibited $O(n^2)$ growth, while divide-and-conquer and heap-based methods (quicksort, mergesort, and heapsort) consistently followed $O(n \log n)$ trends across varying input sizes. Additionally, linear-time

non-comparison sorts—counting sort and radix sort—successfully achieved $O(n)$ scaling on integer data (**VehicleType**), confirming their asymptotic advantage when applicable.

Moreover, the data type and distribution of each column significantly influenced actual sorting speed. Even for algorithms with identical theoretical complexity, sorting floating-point numbers (**TripLength**) was consistently faster than sorting strings (**GantryID_0**) or datetime objects (**DirectionTime_0**), due to differences in comparison cost and memory layout. Notably, columns with high duplicate density (e.g., **VehicleType**) caused standard quicksort to degrade severely, highlighting the critical role of input characteristics in algorithmic performance.

Finally, this experiment underscores that real-world efficiency is not determined by time complexity alone. Implementation details—such as in-place operation, cache locality, recursion depth, and handling of duplicates—combined with hardware-level factors (e.g., CPU cache behavior and branch prediction) substantially affect observed runtime. Therefore, algorithm selection should consider not only theoretical guarantees but also practical implementation and target data properties.